



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

Java Security: Architecture, Primitives and Possible Attacks

Alessandro Marchesano

alessandr.marchesano@unibo.it

Java Cryptography Architecture (JCA)

- The **Java Cryptography Architecture (JCA)** framework within the Java platform provides a set of interfaces and implementations for cryptographic services.
- It defines a set of APIs for digital signatures, message digests, key management, and encryption/decryption, among other cryptographic operations.
- JCA is a core part of the Java Security API and is included in the standard Java Development Kit (JDK).



Safety vs. Security

Safety and Security are two related but distinct concepts:

- **Safety** focuses on preventing accidental failures that could harm the application or the system the program is run on. Examples are *garbage collection, static typing, exceptions handling, thread synchronization, impossibility to handle pointers, strong encapsulation (via private, public, protected)*.
- **Security**, on the other hand, focuses on protecting from intentional attacks by malicious actors.

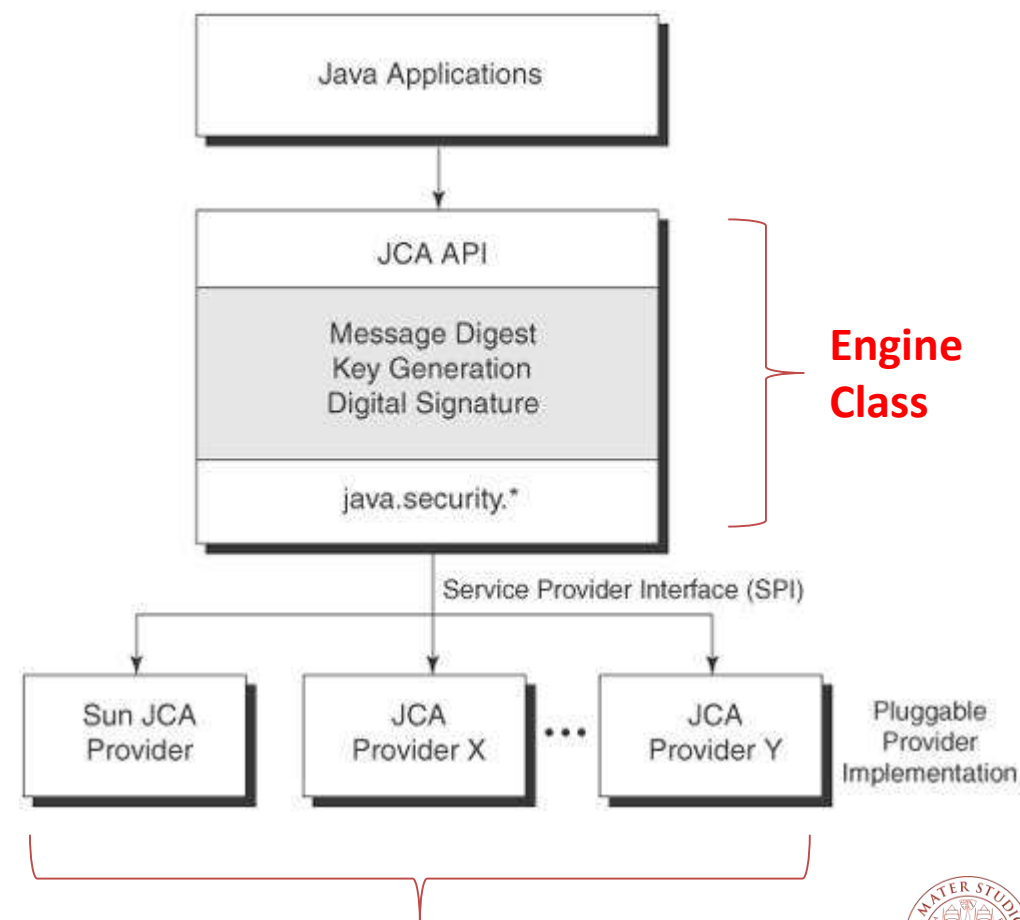
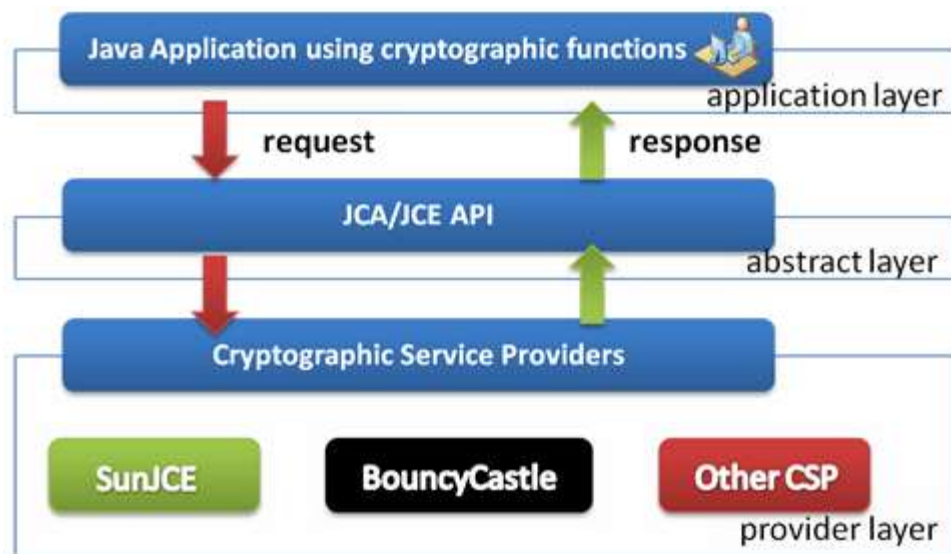


JCA Principles

1. **Implementation independence**: Applications do not need to implement security algorithms. Rather, they can request security services from the Java platform. Security services are implemented in *providers*, which are plugged into the Java platform via a standard interface.
2. **Implementation interoperability**: Providers are interoperable across applications. Specifically, an application is not bound to a specific provider, and a provider is not bound to a specific application.
3. **Algorithm extensibility**: The Java platform includes some built-in providers that implement a basic set of security services that are widely used today, but the Java platform supports the installation of custom providers.



How JCA works



Cryptographic Service Provider (CSP)

- The term *Cryptographic Service Provider* (CSP) refers to a package or set of packages that implement one or more cryptographic services.
- Each JDK installation has one or more providers installed and configured by default. Additional providers may be added statically or dynamically.

```
Java.security.Provider
```



Built-in providers

```
import java.security.*;

public class Test {
    Run main | Debug main | Run | Debug
    public static void main(String[] args) {
        for (Provider provider : Security.getProviders()) {
            System.out.println(provider.getName() + " - " + provider.getInfo());
        }
    }
}
```

SUN - SUN (DSA key/parameter generation; DSA signing; SHA-1, MD5 digests; SecureRandom; X.509 certificates; PKCS12, JKS & DKS keystores; PKIX CertPathValidator; PKIX CertPathBuilder; LDAP, Collection CertStores, JavaPolicy Policy; JavaLoginConfig Configuration)

SunRsaSign - Sun RSA signature provider

SunEC - Sun Elliptic Curve provider

SunJSSE - Sun JSSE provider(PKCS12, SunX509/PKIX key/trust factories, SSLv3/TLSv1/TLSv1.1/TLSv1.2/TLSv1.3/DTLSv1.0/DTLSv1.2)

SunJCE - SunJCE Provider (implements RSA, DES, Triple DES, AES, Blowfish, ARCFOUR, RC2, PBE, Diffie-Hellman, HMAC, ChaCha20, DHKEM, ML-KEM, and HKDF)

SunJGSS - Sun (Kerberos v5, SPNEGO)

SunSASL - Sun SASL provider(implements client mechanisms for: DIGEST-MD5, EXTERNAL, PLAIN, CRAM-MD5, NTLM; server mechanisms for: DIGEST-MD5, CRAM-MD5, NTLM)

XMLDSig - XMLDSig (DOM XMLSignatureFactory; DOM KeyInfoFactory; C14N 1.0, C14N 1.1, Exclusive C14N, Base64, Enveloped, XPath, XPath2, XSLT TransformServices)

SunPCSC - Sun PC/SC provider

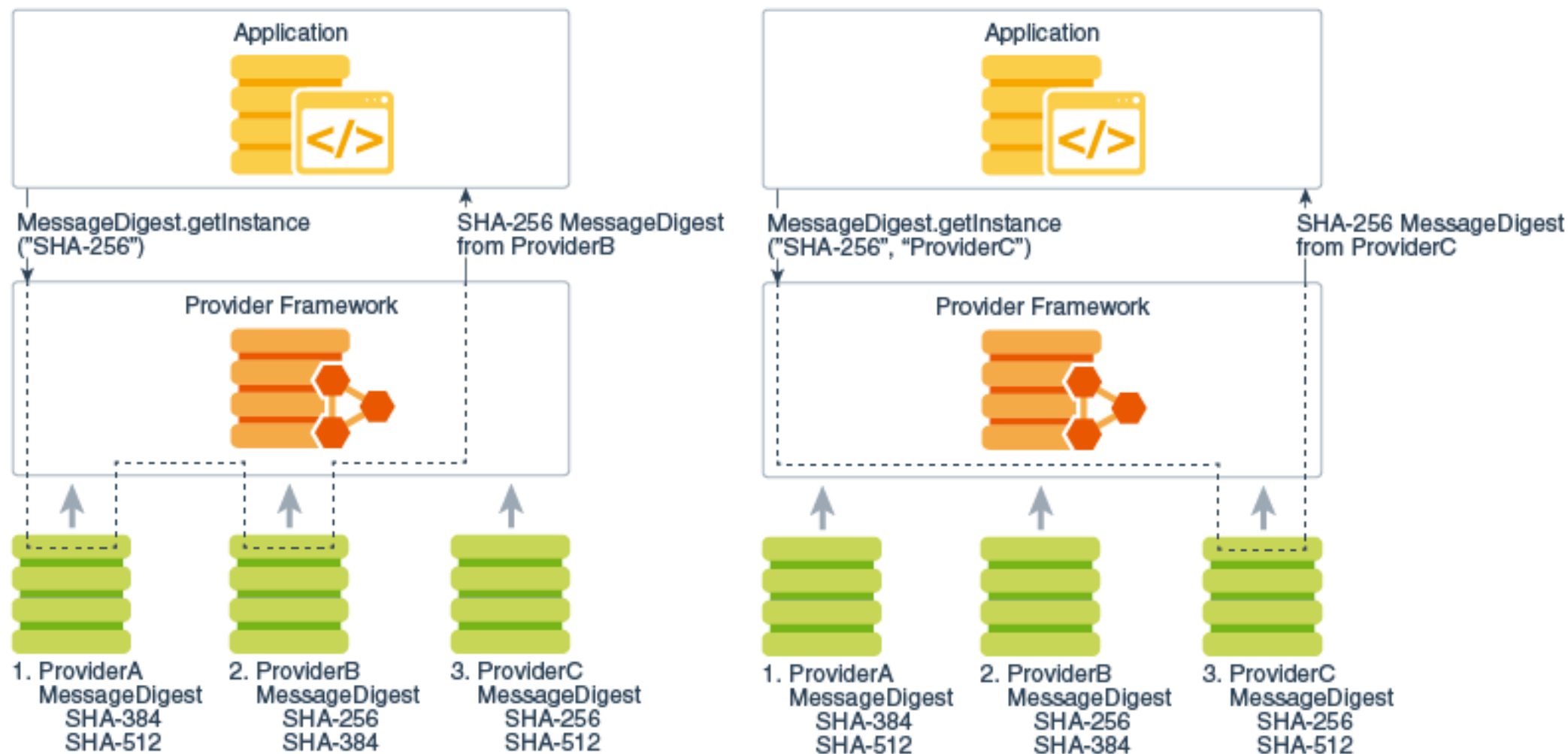
JdkLDAP - JdkLDAP Provider (implements LDAP CertStore)

JdkSASL - JDK SASL provider(implements client and server mechanisms for GSSAPI)

SunMSCAPI - Sun's Microsoft Crypto API provider

SunPKCS11 - Unconfigured and unusable PKCS11 provider

How Providers work



How to request services

```
static EngineClassName getInstance(String algorithm)  
    throws NoSuchAlgorithmException
```

```
static EngineClassName getInstance(String algorithm, String provider)  
    throws NoSuchAlgorithmException, NoSuchProviderException
```

```
static EngineClassName getInstance(String algorithm, Provider provider)  
    throws NoSuchAlgorithmException
```

where EngineClassName is the desired engine type (for example, Signature, MessageDigest, or Cipher). For example:

```
Signature sig = Signature.getInstance("SHA256withRSA");  
MessageDigest md = MessageDigest.getInstance("SHA-256")
```



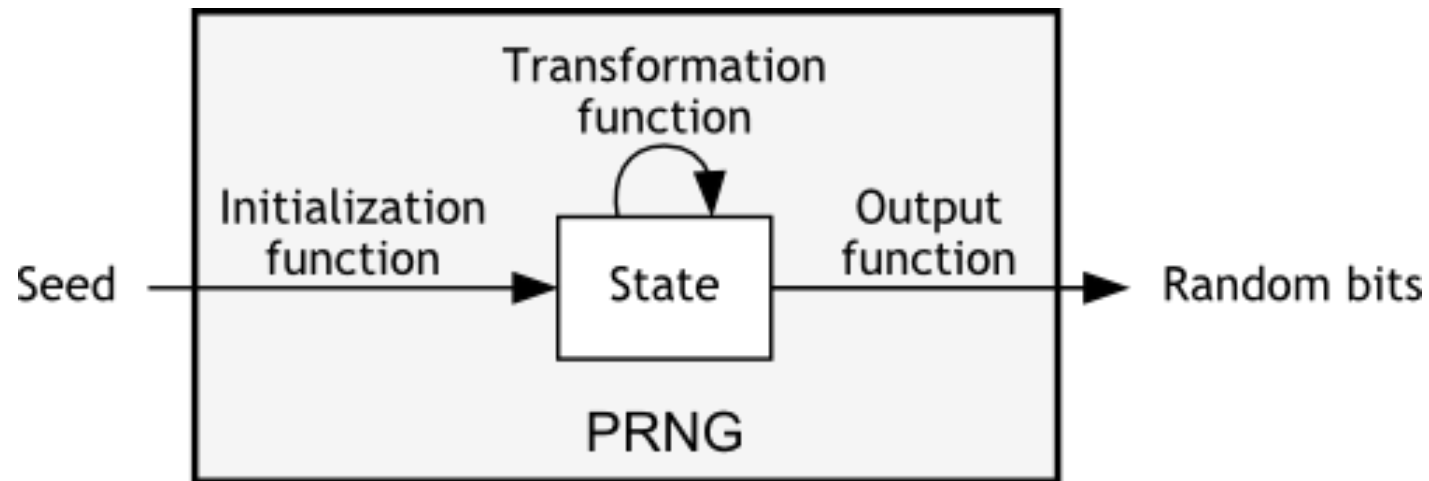
What services (engine classes) can we request?

- SecureRandom
- MessageDigest
- Signature
- Cipher
- Mac
- KeyFactory
- SecretKeyFactory
- KeyPairGenerator
- KeyGenerator
- KeyAgreement
- AlgorithmParameters
- AlgorithmParameterGenerator
- KeyStore
- CertificateFactory
- CertPathBuilder
- CertPathValidator
- CertStore



SecureRandom

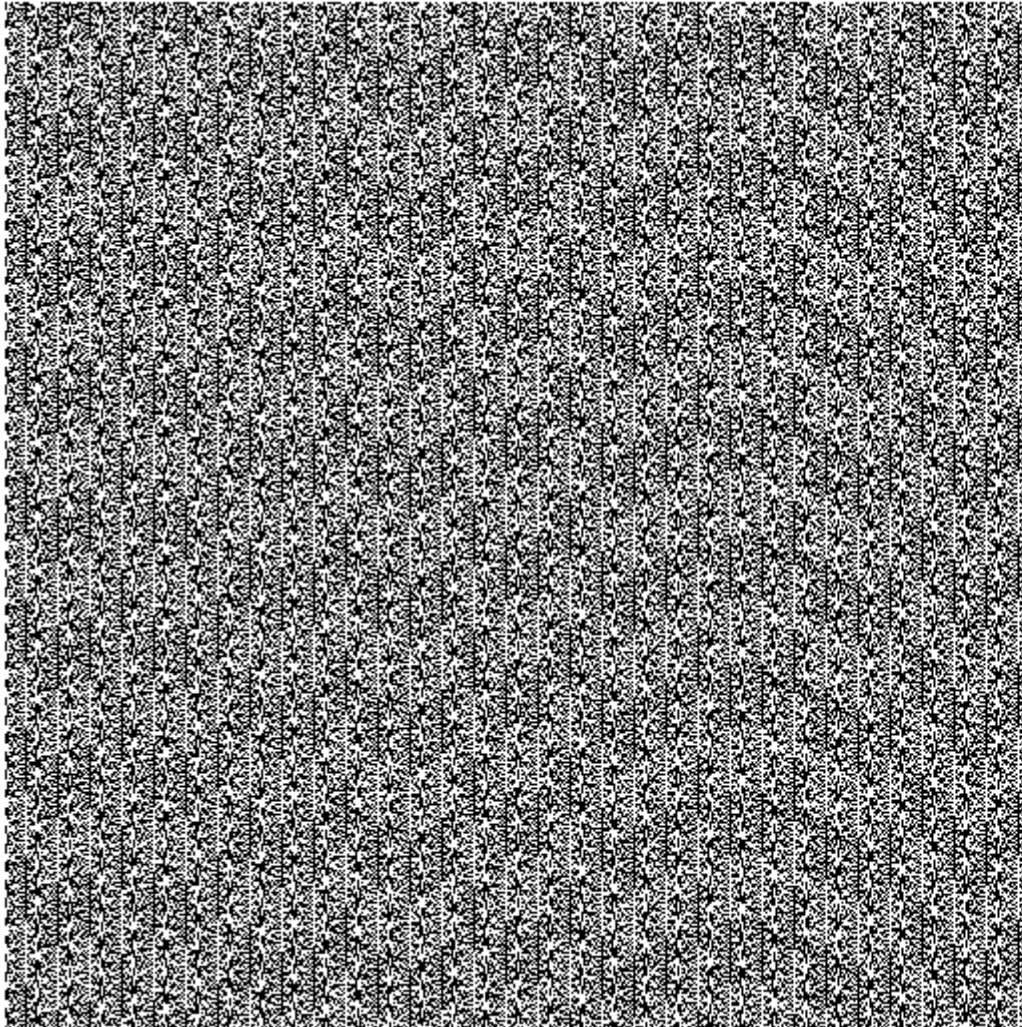
- The SecureRandom class is an engine class that provides **cryptographically strong random numbers**, using PRNGs, OS-native sources, or a combination of both.
- They are used to instantiate ciphers, MACs, algorithm parameters or in application-specific scenarios.



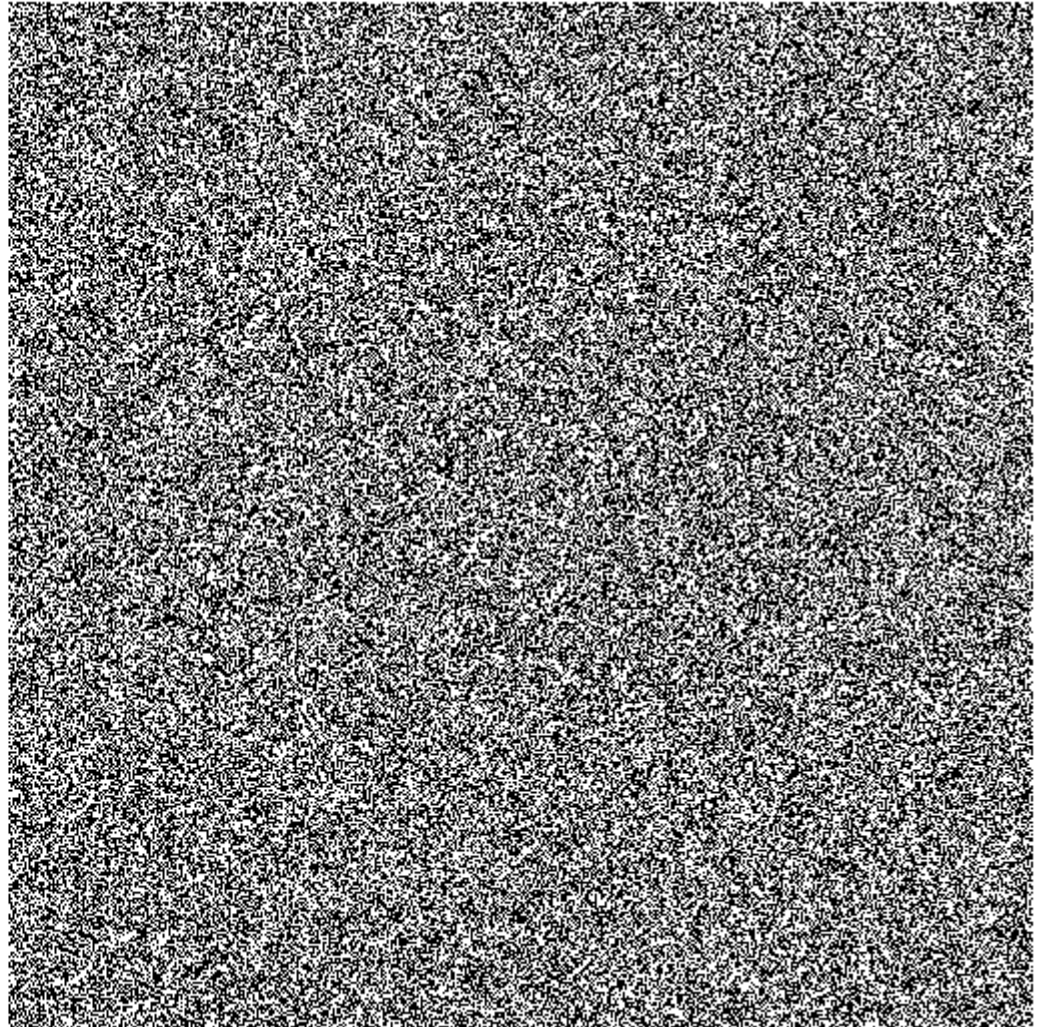
Random vs SecureRandom

- The classes defined in Random are **NOT** cryptographically strong, and the numbers chosen are not completely random because a definite mathematical algorithm is used to select them. Therefore, it is not safe to use this class for tasks that require a high level of security.
- Instead, SecureRandom provides a cryptographically strong RNG... if used properly.





Random class algorithm



SecureRandom class algorithm

Getting a SecureRandom object

We can get a SecureRandom object in three ways:

1. `new SecureRandom()` - returns a new SecureRandom object from the first provider that supports a SecureRandom RNG algorithm. SHA1PRNG from SUN if none found.
2. `SecureRandom.getInstance("algorithmName")` - to get a specific implementation of SecureRandom
3. `SecureRandom.getInstanceStrong()` - to obtain a strong implementation as defined by the `securerandom.strongAlgorithms` property of the `java.security.Security` class.

Possible algorithms:

- NativePRNG
- NativePRNGBlocking
- NativePRNGNonBlocking
- PKCS11
- DRBG
- SHA1PRNG
- Windows-PRNG



Using a SecureRandom object

Available methods:

- `public void setSeed(long seed);` - sets the seed in the SecureRandom PRNG.
- Overridden `java.util.Random` methods:
 - `public int nextInt();` Securely generates a random integer.
 - `public int nextBoolean();` Securely generates a random boolean.
 - `public byte[] generateSeed(int numBytes);` Returns the given number of seed bytes.
 - `public void nextBytes(byte[] bytes);` Securely generates a user-specified number of random bytes



How to deal with seeds

- The randomness of the bits produced by the SecureRandom object depends on the randomness of the seed bits.
- At the beginning, the objects are not seeded. If, after the instantiation, *nextBytes* (or whatever operation) is called, then the PRNG is seeded using a secure mechanism provided by the underlying operating system.
- If *setSeed* is called before a call to *nextBytes*, then the internal seeding mechanism is bypassed, and only the provided seed is used to generate random numbers.
- At any time, a SecureRandom object may be re-seeded using one of the *setSeed* or *reseed* methods. The given seed for *setSeed* supplements, rather than replaces, the existing seed; therefore, repeated calls are guaranteed never to reduce randomness.



SHA1PRNG with and without seed initialization

```
SecureRandom sr1, sr2;  
try{  
    sr1 = SecureRandom.getInstance(algorithm: "SHA1PRNG");  
    sr2 = SecureRandom.getInstance(algorithm: "SHA1PRNG");  
}catch(NoSuchAlgorithmException e){  
    System.out.println("Algorithm not found: " + e.getMessage());  
    return;  
}
```

Results:

```
1 (SHA1): 377998523  
2 (SHA1): 1475112452
```

```
SecureRandom sr1, sr2;  
try{  
    sr1 = SecureRandom.getInstance(algorithm: "SHA1PRNG");  
    sr2 = SecureRandom.getInstance(algorithm: "SHA1PRNG");  
}catch(NoSuchAlgorithmException e){  
    System.out.println("Algorithm not found: " + e.getMessage());  
    return;  
}  
  
sr1.setSeed(seed: 1);  
sr2.setSeed(seed: 1);
```

Results:

```
1 (SHA1): -1765061395  
2 (SHA1): -1765061395
```



SHA1PRNG with automatic seed initialization

```
SecureRandom sr1, sr2;  
try{  
    sr1 = SecureRandom.getInstance(algorithm: "SHA1PRNG");  
    sr2 = SecureRandom.getInstance(algorithm: "SHA1PRNG");  
}catch(NoSuchAlgorithmException e){  
    System.out.println("Algorithm not found: " + e.getMessage());  
    return;  
}  
  
sr1.nextBytes(new byte[16]);  
sr2.nextBytes(new byte[16]);  
sr1.setSeed(seed: 1);  
sr2.setSeed(seed: 1);
```

Results:

```
1 (SHA1): 595659231  
2 (SHA1): -1418020312
```

However, SHA1PRNG is NOT considered secure.

DRBG (1)

```
SecureRandom sr1 = new SecureRandom();  
SecureRandom sr2 = new SecureRandom();  
  
System.out.println("1 (" + sr1.getAlgorithm() + "): " + Integer.toString(sr1.nextInt()));  
System.out.println("2 (" + sr2.getAlgorithm() + "): " + Integer.toString(sr2.nextInt()));
```

```
1 (DRBG): -1277048928  
2 (DRBG): -296944548
```



DRBG (2)

```
SecureRandom sr1 = new SecureRandom();  
SecureRandom sr2 = new SecureRandom();  
  
sr1.setSeed(seed: 1);  
sr2.setSeed(seed: 1);  
  
System.out.println("1 (" + sr1.getAlgorithm() + "): " + Integer.toString(sr1.nextInt()));  
System.out.println("2 (" + sr2.getAlgorithm() + "): " + Integer.toString(sr2.nextInt()));
```

First Execution:

```
1 (DRBG): 417885477  
2 (DRBG): -311928207
```

Second Execution:

```
1 (DRBG): 417885477  
2 (DRBG): -311928207
```



Strong Instance

```
SecureRandom sr1, sr2;  
try{  
    sr1 = SecureRandom.getInstanceStrong();  
    sr2 = SecureRandom.getInstanceStrong();  
}catch(NoSuchAlgorithmException e){  
    System.out.println("Strong algorithm not found: " + e.getMessage());  
    return;  
}  
  
System.out.println("1 (" + sr1.getAlgorithm() + "): " + Integer.toString(sr1.nextInt()));  
System.out.println("2 (" + sr2.getAlgorithm() + "): " + Integer.toString(sr2.nextInt()));
```

```
1 (Windows-PRNG): -1824239806  
2 (Windows-PRNG): 402828754
```

```
SecureRandom sr1, sr2;  
try{  
    sr1 = SecureRandom.getInstanceStrong();  
    sr2 = SecureRandom.getInstanceStrong();  
}catch(NoSuchAlgorithmException e){  
    System.out.println("Strong algorithm not found: " + e.getMessage());  
    return;  
}  
  
sr1.setSeed(seed: 1);  
sr2.setSeed(seed: 1);
```

```
1 (Windows-PRNG): 480279798  
2 (Windows-PRNG): -1852413068
```

```
1 (Windows-PRNG): 399769140  
2 (Windows-PRNG): -522844589
```

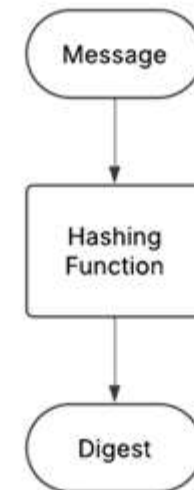
What did we learn?

1. Always use SecureRandom. **DO NOT** use Random class to generate random values.
2. SHA1PRNG is **NOT** considered secure and should not be used.
3. Let the framework auto-initialize the seed. Do not use setSeed or use it only after a nextBytes (or other) call.
4. If you need to manually set a seed, generate a cryptographically secure one.
5. Even though getInstanceStrong appears to be the best option, **it actually depends**.

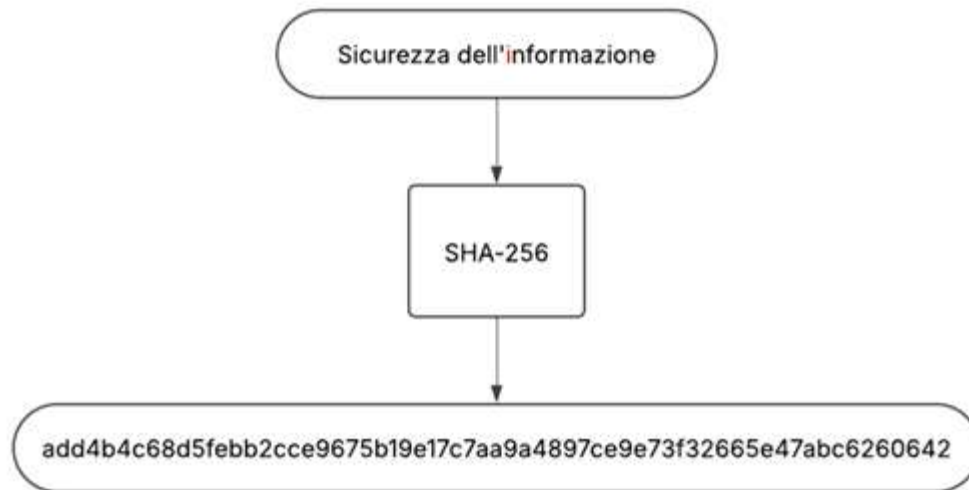


MessageDigest

- To avoid modifications of the information in the unsafe channel, we need a hashing function that takes in input a string of information of any length and outputs a unique fingerprint of a fixed length.
- Inputting the same string of information into a hashing function always outputs the same fingerprint.
- The modification of a single bit must completely change the output fingerprint.



MessageDigest



Getting a MessageDigest object

We can get a MessageDigest object with:

```
MessageDigest.getInstance("algorithmName")
```

It thus does not need further initialization.

Possible algorithms:

- ~~MD2~~
- ~~MD5~~
- ~~SHA-1~~
- SHA-224
- **SHA-256**
- SHA-384
- SHA-512
- SHA-512/224
- SHA-512/256



Using a MessageDigest object

The MessageDigest class provides three methods (and many overloading implementations) to manage digest creation:

- `public final void update(ByteBuffer input);`
Updates the input buffer by initializing it or appending `input` to it.
- `public final void reset();`
Resets the input buffer.
- `public final byte[] digest();`
Computes and returns the hash/digest/fingerprint of the input. It automatically resets the input buffer.



Hashing a message

```
try{
    String message = "Sicurezza dell'Informazione";
    byte[] messageBytes = message.getBytes();
    ByteBuffer buf = ByteBuffer.wrap(messageBytes);

    MessageDigest md = MessageDigest.getInstance("SHA-256");
    md.update(buf);

    byte[] digest = md.digest();
    String hexDigest = HexFormat.of().formatHex(digest);

    System.out.println("Message: " + message);
    System.out.println("Digest: " + hexDigest);
}catch(NoSuchAlgorithmException e){
    System.out.println("Algorithm not found: " + e.getMessage());
}
```

Message: Sicurezza dell'Informazione

Digest: 213f3bd22a326a6820f68bfa69f26ac24fdf2432454e178250b9ff94d0a35c86



Hashing a message in more steps

```
try {  
    MessageDigest md = MessageDigest.getInstance("SHA-256");  
    md.update("Sicurezza ".getBytes());  
    md.update("dell' ".getBytes());  
    md.update("Informazione".getBytes());  
  
    byte[] digest = md.digest();  
    System.out.println("Digest: " + HexFormat.of().formatHex(digest));  
} catch (NoSuchAlgorithmException e) {  
    e.printStackTrace();  
}
```

Digest: 213f3bd22a326a6820f68bfa69f26ac24fdf2432454e178250b9ff94d0a35c86



Cipher

The *Cipher* class provides the functionality of a cryptographic cipher used for encryption and decryption.

Symmetric encryption requires that the same key is used both for encrypting and decrypting.

The participants must preemptively agree on the key.

Two types of symmetric ciphers exist:

- Block ciphers
- Stream ciphers

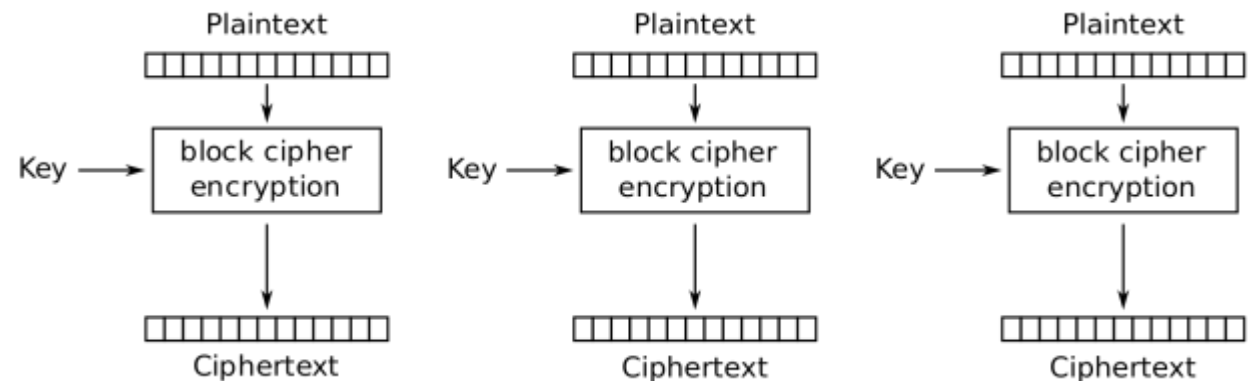


Block Ciphers

- Block ciphers process **entire blocks at a time**, usually many bytes in length.
- If there is not enough data to make a complete input block, **the data must be padded**: that is, before encryption, dummy bytes must be added to make a multiple of the cipher's block size. These bytes are then stripped off during the decryption phase.

Examples of block ciphers are:

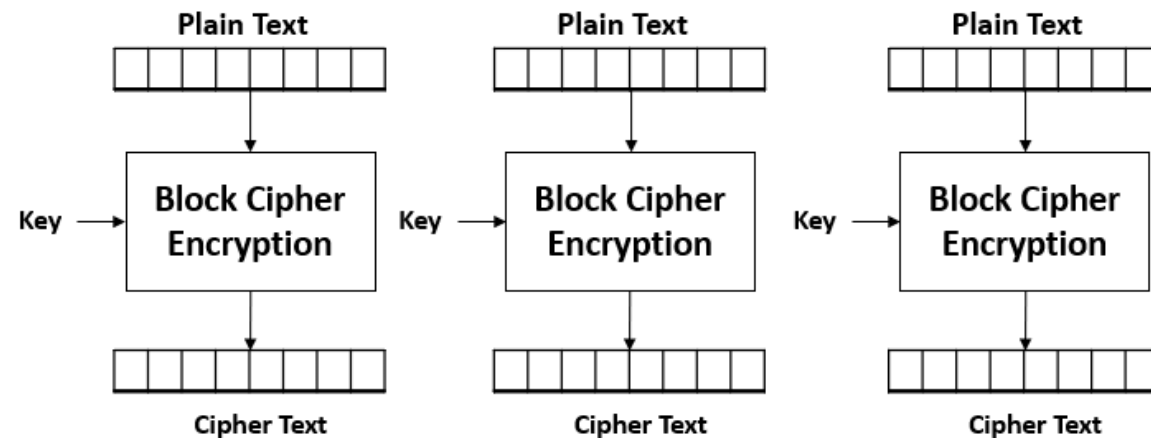
- AES-EBC (Electronic Code Book)
- AES-CBC (Cipher Block Chaining)
- AES-CFB (Cipher Feedback Mode)
- AES-CTR (Counter Mode)*
- AES-GCM (Galois Counter Mode)*

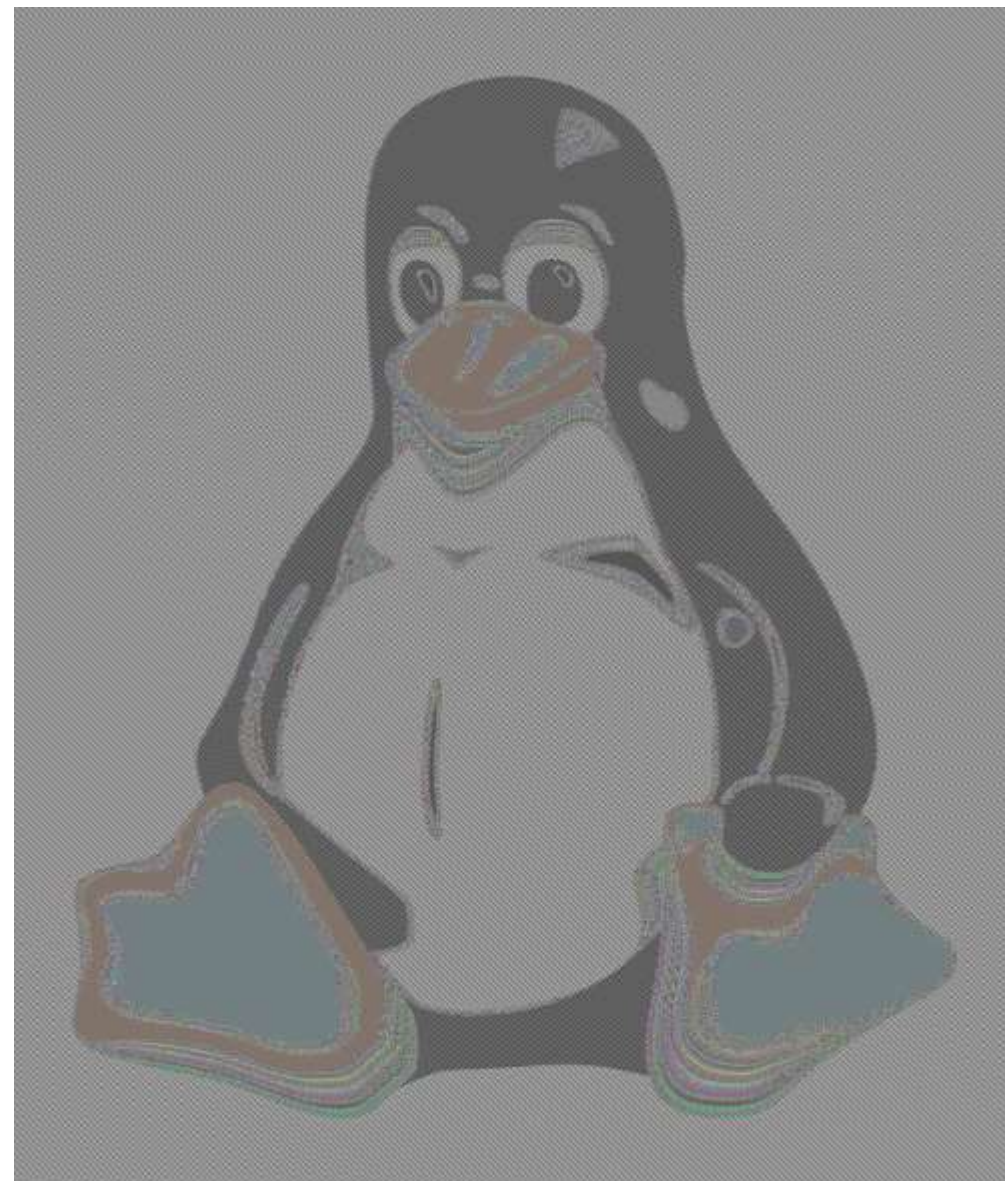


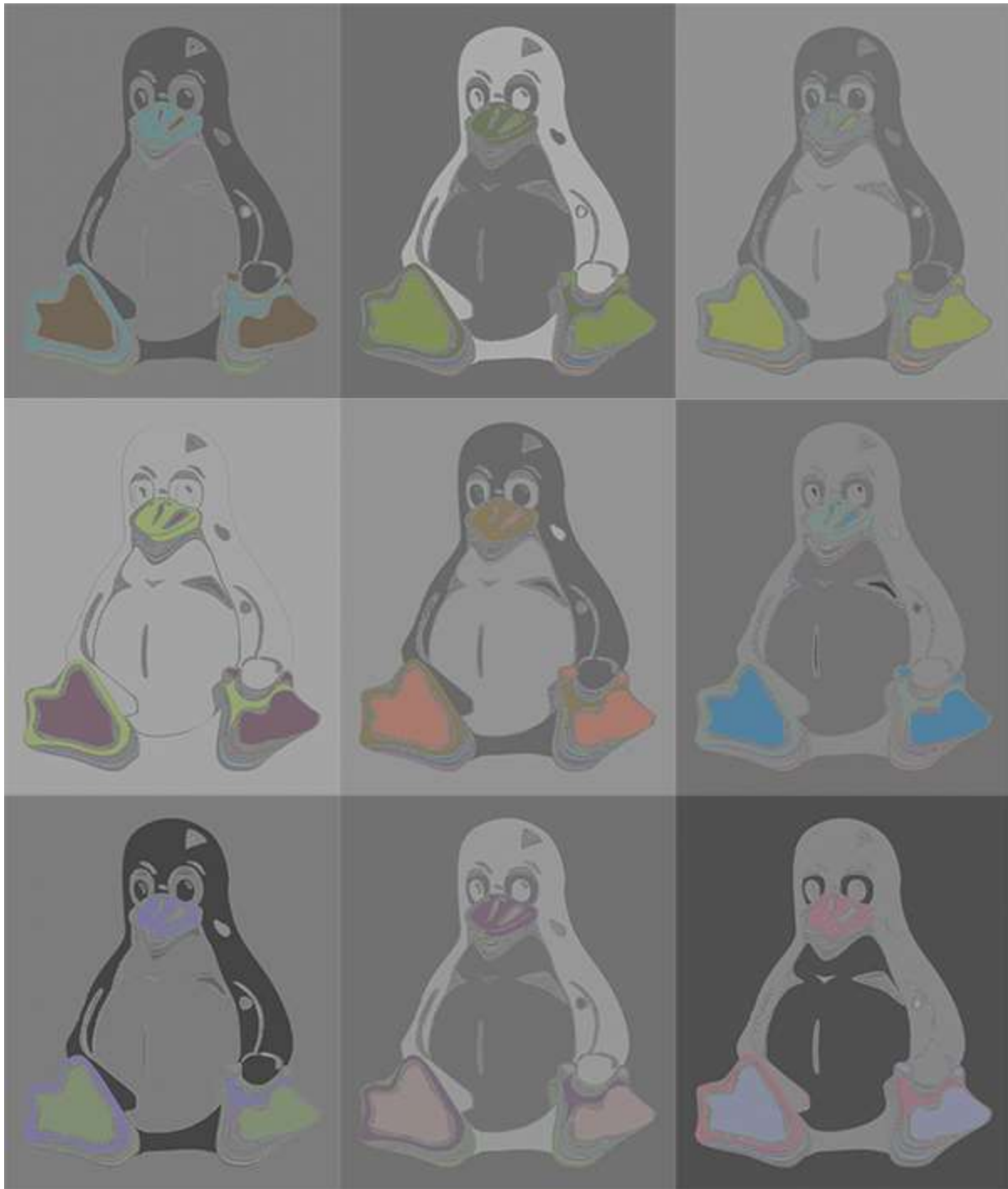
Why we should not use ECB

ECB mode is considered insecure because it encrypts identical plaintext blocks into identical ciphertext blocks, revealing patterns in the data.

Because each block is encrypted independently with the same key, identical plaintext blocks produce identical ciphertext blocks. This means that structured data (like images or formatted text) will still show its structure after encryption.





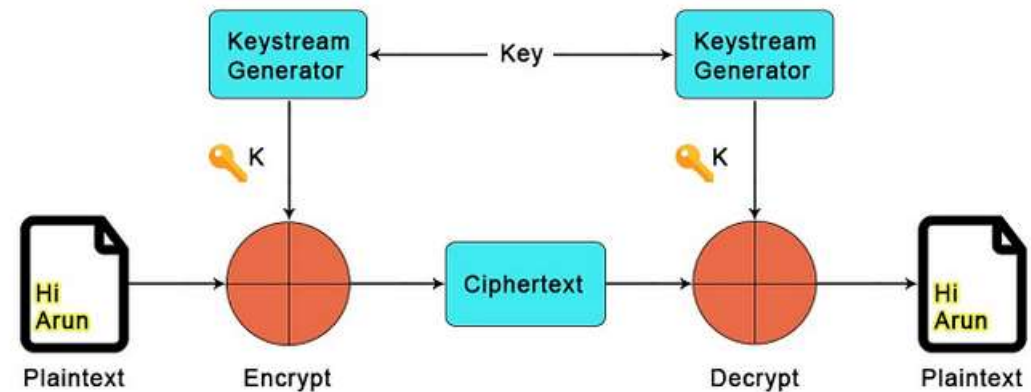


Stream Ciphers

- Stream ciphers process incoming data one small unit (typically a byte or even a bit) at a time. This allows for ciphers to process an arbitrary amount of data without padding.
- Depending on the particular algorithm, other parameters may be needed (like nonces or counter values).

Examples of Stream Ciphers are:

- RC4 (Rivest Cipher 4)
- Salsa20
- ChaCha20
- *AES-CTR (Counter Mode)*
- *AES-GCM (Galois Counter Mode)*



Getting a Cipher object

- An instance of `Cipher` is obtainable once again through

```
public static Cipher getInstance(String transformation);
```

- A transformation is of the form "*algorithm/mode/padding*"
- Examples:

//block

```
Cipher cipher = Cipher.getInstance("AES");
```

```
Cipher cipher = Cipher.getInstance("AES/ECB/PKCS5Padding");
```

//stream

```
Cipher cipher = Cipher.getInstance("ChaCha20");
```



Initializing a Cipher object

A Cipher object must be initialized for one of four modes:

- ENCRYPT_MODE
- DECRYPT_MODE
- WRAP_MODE
- UNWRAP_MODE

To initialize a `Cipher` object, call one of the following `init` methods:

- `public void init(int opmode, Key key);`
- `public void init(int opmode, Key key, SecureRandom random);`
- `public void init(int opmode, Key key, AlgorithmParameterSpec params);`
- `public void init(int opmode, Key key, AlgorithmParameterSpec params, SecureRandom random);`



Generating Keys

- Of course, to use ciphers, we need cryptographic key material. To generate keys, we need a `KeyGenerator` object.

- With no one's surprise, available via:

```
public static KeyGenerator getInstance(String algorithm);  
KeyGenerator kg = KeyGenerator.getInstance("AES");
```

- Initialize the object using:

```
public void init(int keySize);  
kg.init(256);
```

- Obtain a secret key:

```
public SecretKey generateKey();  
SecretKey sk = kg.generateKey();
```



About Cipher parameters...

- If a Cipher object that requires parameters (for example, an IV) is initialized for **encryption**, and no parameters are supplied to the `init` method, the underlying cipher implementation is supposed to supply the required parameters itself, generating random parameters or using a default, provider-specific set of parameters.
- However, if a Cipher object that requires parameters is initialized for **decryption**, and no parameters are supplied to the `init` method, an exception will be raised.
- **The same parameters that were used for encryption must be used for decryption.**
- When a Cipher is initialized, **it loses all previously acquired state**. In other words, initializing a Cipher is equivalent to creating a new instance of that Cipher and initializing it.

When you don't specify all parameters during initialization, Cipher method `getParameters()` could be useful.



What parameters do we need?

- Depending on the mode of operations, some additional information might be required:
 - **ECB**: no additional information.
 - **CBC, OFB, CFB**: require an Initialization Vector.
 - **CTR**: requires an Initialization Vector and No padding.
 - **GCM**: requires an Initialization Vector, the MAC dimension, and No padding.
 - **ChaCha20**: requires a nonce and a counter.



How do we generate parameters?

- First, create a PRNG and fill a byte array:

```
SecureRandom sr = SecureRandom.getInstanceStrong();  
byte[] iv = new byte[16];  
sr.nextBytes(iv);
```

- To provide an Initialization Vector:

```
IvParameterSpec spec = new IvParameterSpec(iv);
```

- For GCM specifically, we also need to provide the length in bit of the MAC.

```
int macLen = 128;  
GCMParameterSpec spec = new GCMParameterSpec(macLen, iv);
```

- In the signature of the `init` method, `AlgorithmParameterSpec` is a superclass of both `IvParameterSpec` and `GCMParameterSpec`.



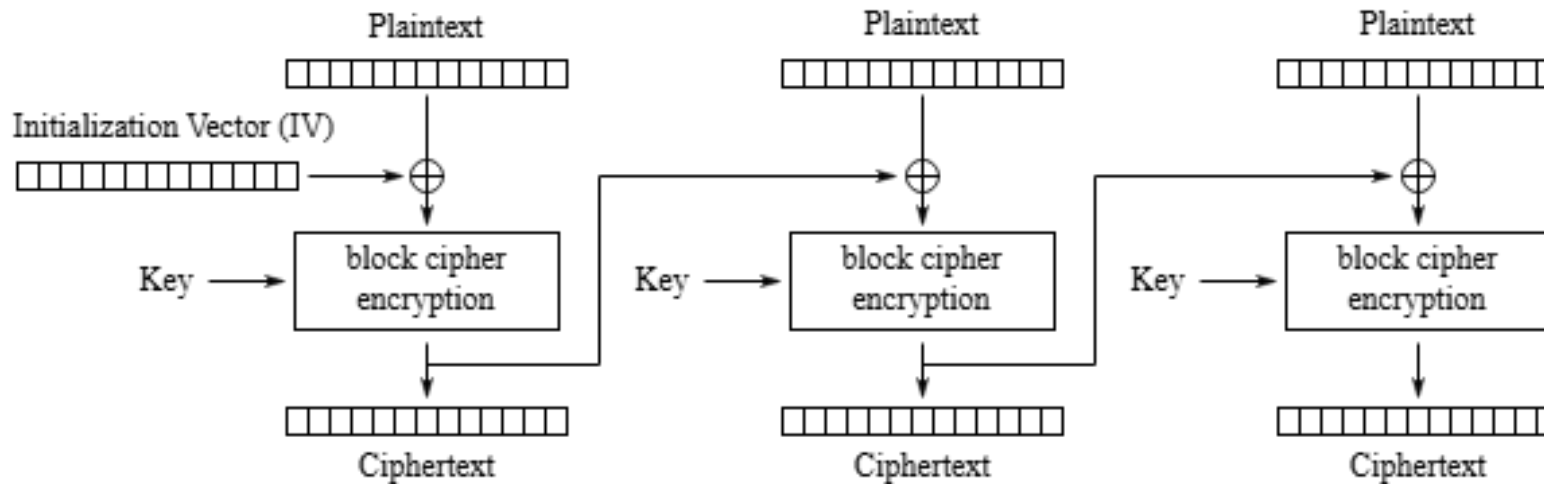
Stream Ciphers Parameters

- To employ stream ciphers like **ChaCha20** we must supply additional parameters exactly as we did for modes of operations.
- **ChaCha20**, for example, must be supplied with a **nonce** and a **counter** variable.

```
byte[] nonce = new byte[12];  
sr.nextBytes(nonce);  
int counter = sr.nextInt();  
ChaCha20ParameterSpec spec = new  
    ChaCha20ParameterSpec(nonce, counter);
```



IV mishandling – CBC (1)



Cipher Block Chaining (CBC) mode encryption

$$\begin{cases} C_i = E_{key}(P_i \oplus C_{i-1}) \\ C_0 = IV \end{cases}$$

- What if we use a constant IV?
- What if we use a predictable IV (such as a counter)?

IV mishandling – CBC (2)

Recall the following relation for the CBC mode :

$$C_i = E_{key}(P_i \oplus C_{i-1})$$

If the attacker has predicted the IV that will be used for the next encryption operation (IV'), he can submit his guess (P_g) of the plaintext P_i for which he knows the encryption result C_i .

To do so, he computes the following plaintext value :

$$P' = C_{i-1} \oplus P_g \oplus IV'$$

The victim will encrypt this value, resulting in :

$$\begin{aligned} C' &= E_{key}(P' \oplus IV') \\ &= E_{key}(C_{i-1} \oplus P_g \oplus IV' \oplus IV') \\ &= E_{key}(C_{i-1} \oplus P_g) \end{aligned}$$

If the guess is correct ($P_g = P_i$) then the resulting ciphertext should match the previously known one ($C' = C_i$).



IV mishandling – CBC (3)

Imagine a protocol that encrypts each network packet by using the last encrypted block as the new IV. Also suppose that transmitted data is limited to “Yes” or “No”.

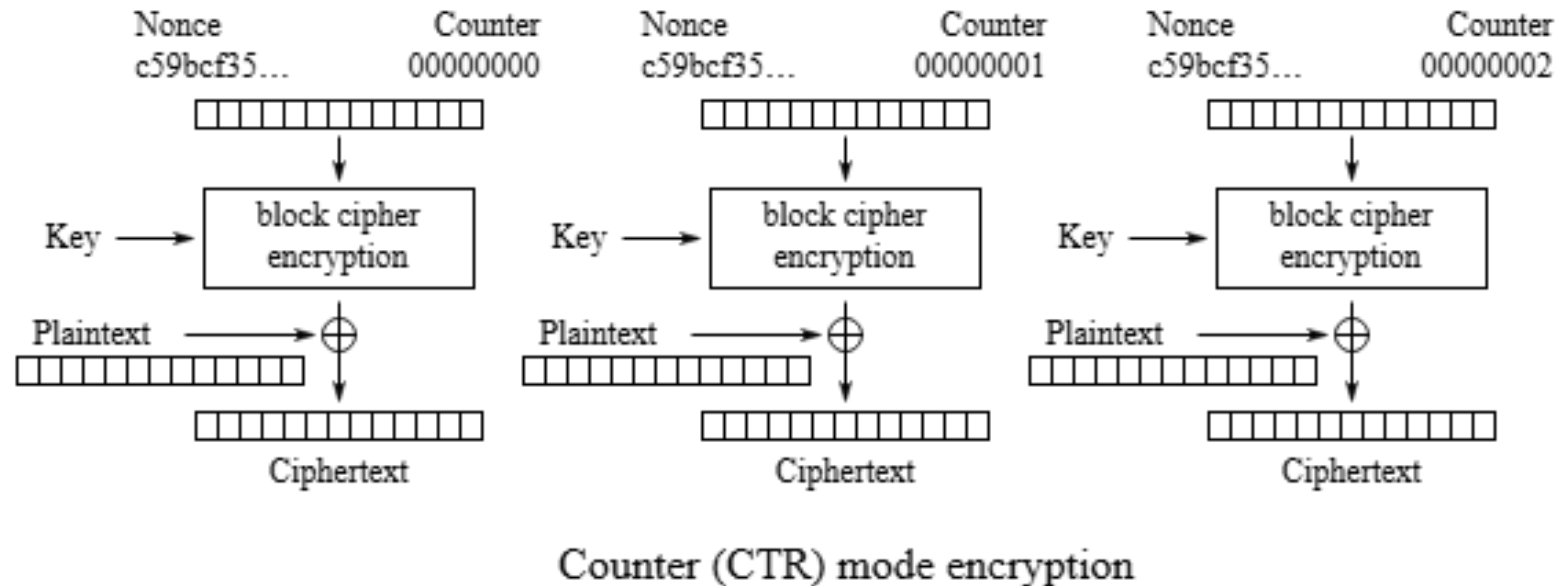
An attacker who intercepted the communication might try to guess the message by submitting guesses until the right value is found.

When the guess is right, the encryption result matches the intercepted one.

Although this example might seem far-fetched, this is roughly what is done by the BEAST attack, allowing it to decrypt traffic in TLS 1.0 and below.



IV mishandling – CTR (1)



- What if we use a constant IV?
- What if we use a reused IV?

IV mishandling – CTR (2)

Just like for CBC mode, you should not reuse a key/IV pair.

If the same IV is used twice, the same keystream will be produced. The attacker can compute the XOR operation of the two plaintexts, allowing him to decrypt them using **crib dragging**.

$$C_1 = P_1 \oplus KS$$

$$C_2 = P_2 \oplus KS$$

$$C_1 \oplus C_2 = P_1 \oplus P_2$$



IV mishandling – CTR (3)

The solution could be to generate the IV randomly, but...

The key takeaway is that if you generate an n -bit random value, the probability of obtaining a collision approximates 50% after just $2^{(n/2)}$ generations, because of the birthday paradox.

If n is sufficiently large, the probability of collision is negligible for a longer time, but keep in mind that a single collision is enough to break confidentiality.

The best solution is to build an IV as a *nonce* + a *counter*, ensuring unpredictability and uniqueness.



Using a Cipher Object

- Finally, when the cipher object has been initialized, we can start encrypting or decrypting.

- For one-shot encryption/decryption we use:

```
public final byte[] doFinal(byte[] input);
```

- For multi-part encryption/decryption, assuming we have n parts:

```
public final byte[] update(byte[] input); // for n-1 parts
```

```
public final byte[] doFinal(byte[] input); // for last part
```



Ciphers Example – Block Ciphers, CBC

```
// Encryption
SecureRandom sr = new SecureRandom();
KeyGenerator kg = KeyGenerator.getInstance("AES");
kg.init(256, sr);
SecretKey key = kg.generateKey();

byte[] iv = new byte[16];
sr.nextBytes(iv);
IvParameterSpec ivSpec = new IvParameterSpec(iv);

Cipher cipher = Cipher.getInstance("AES/CBC/PKCS5Padding");
cipher.init(Cipher.ENCRYPT_MODE, key, ivSpec);

byte[] plaintext = "Sicurezza dell'Informazione".getBytes();
byte[] ciphertext = cipher.doFinal(plaintext);
String encodedCiphertext = HexFormat.of().formatHex(ciphertext);
System.out.println("Ciphertext: " + encodedCiphertext);

// Decryption
cipher.init(Cipher.DECRYPT_MODE, key, ivSpec);
byte[] decryptedPlaintextBytes = cipher.doFinal(ciphertext);
String decryptedPlaintext = new String(decryptedPlaintextBytes);
System.out.println("Decrypted Plaintext: " + decryptedPlaintext);
```

Ciphertext: 726bca29fafcc4a306696f620143839265ad5e238a0a0d9ee4bb21d6f4d0cd9c
Decrypted Plaintext: Sicurezza dell'Informazione

Ciphers Example (2) – Stream Ciphers, ChaCha20

```
// Encryption
SecureRandom sr = new SecureRandom();
KeyGenerator kg = KeyGenerator.getInstance("AES");
kg.init(256, sr);
SecretKey key = kg.generateKey();

byte[] nonce = new byte[12];
sr.nextBytes(nonce);
int counter = sr.nextInt();
ChaCha20ParameterSpec paramSpec = new ChaCha20ParameterSpec(nonce, counter);

Cipher cipher = Cipher.getInstance("CHACHA20");
cipher.init(Cipher.ENCRYPT_MODE, key, paramSpec);

byte[] plaintext = "Sicurezza dell'Informazione".getBytes();
byte[] ciphertext = cipher.doFinal(plaintext);
String encodedCiphertext = HexFormat.of().formatHex(ciphertext);
System.out.println("Ciphertext: " + encodedCiphertext);

// Decryption
cipher.init(Cipher.DECRYPT_MODE, key, paramSpec);
byte[] decryptedPlaintextBytes = cipher.doFinal(ciphertext);
String decryptedPlaintext = new String(decryptedPlaintextBytes);
System.out.println("Decrypted Plaintext: " + decryptedPlaintext);
```

Ciphertext: e5282a89811220a707ed09279bdd787984f084b9df07994199174a

Decrypted Plaintext: Sicurezza dell'Informazione



Other Cipher-based classes - CipherInputStream

This class is a `FilterInputStream` that encrypts or decrypts the data passing through it. It is composed of an `InputStream`. `CipherInputStream` represents a secure input stream into which a `Cipher` object has been interposed. The read methods of `CipherInputStream` return data that are read from the underlying `InputStream` but have additionally been processed by the embedded `Cipher` object.

```
try (FileInputStream fis = new FileInputStream("/tmp/a.txt");
    CipherInputStream cis = new CipherInputStream(fis, cipher1);
    FileOutputStream fos = new FileOutputStream("/tmp/b.txt")) {
    byte[] b = new byte[8];
    int i = cis.read(b);
    while (i != -1) {
        fos.write(b, 0, i);
        i = cis.read(b);
    }
}
```

Other Cipher-based classes - CipherOutputStream

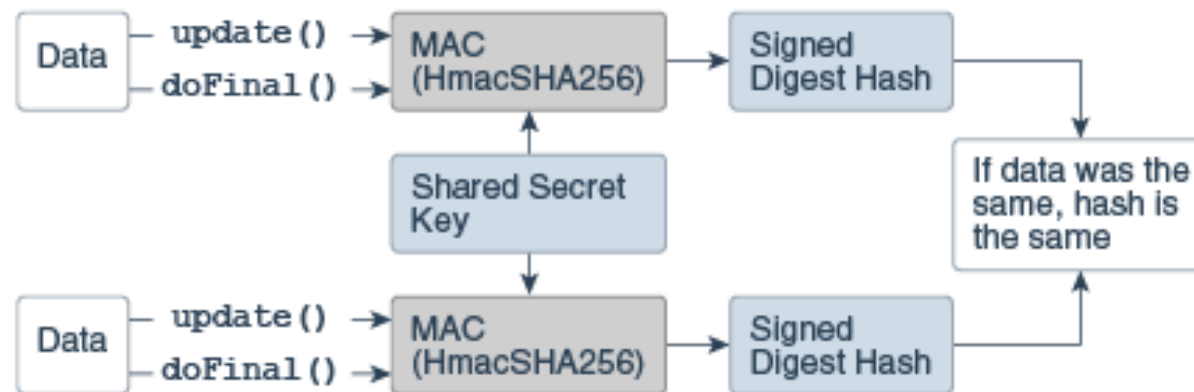
This class is a `FilterOutputStream` that encrypts or decrypts the data passing through it. It is composed of an `OutputStream`, or one of its subclasses, and a `Cipher`. `CipherOutputStream` represents a secure output stream into which a `Cipher` object has been interposed. The write methods of `CipherOutputStream` first process the data with the embedded `Cipher` object before writing them out to the underlying `OutputStream`.

```
try (FileInputStream fis = new FileInputStream("/tmp/a.txt");
    FileOutputStream fos = new FileOutputStream("/tmp/b.txt");
    CipherOutputStream cos = new CipherOutputStream(fos, cipher1)) {
    byte[] b = new byte[8];
    int i = fis.read(b);
    while (i != -1) {
        cos.write(b, 0, i);
        i = fis.read(b);
    }
}
```

Message Authentication Code (MAC)

Similar to a MessageDigest, a MAC provides a way to check the integrity of information transmitted over or stored in an unreliable medium but includes a secret key in the calculation.

A MAC mechanism that is based on cryptographic hash functions is referred to as HMAC. HMAC can be used with any cryptographic hash function, e.g., SHA-256, in combination with a secret shared key.



Creating a Mac object

- Mac objects are obtained by using one of the `Mac getInstance()` static factory methods.

```
Mac.getInstance("algorithmName");
```

Examples of Mac algorithms are:

- HmacMD5
- HmacSHA1
- HmacSHA224
- HmacSHA256
- HmacSHA512



Initializing a Mac object

A Mac object is always initialized with a (secret) key and may optionally be initialized with a set of parameters, depending on the underlying MAC algorithm.

```
public void init(Key key);
```

```
public void init(Key key, AlgorithmParameterSpec params);
```



Using a Mac object

- A MAC can be computed in one step or in multiple. A multiple-part operation is useful if you do not know in advance how long the data is going to be, or if the data is too long to be stored in memory all at once.

- For one-shot operations we use:

```
public byte[] doFinal(byte[] input);
```

- For multi-part operations, assuming we have n parts:

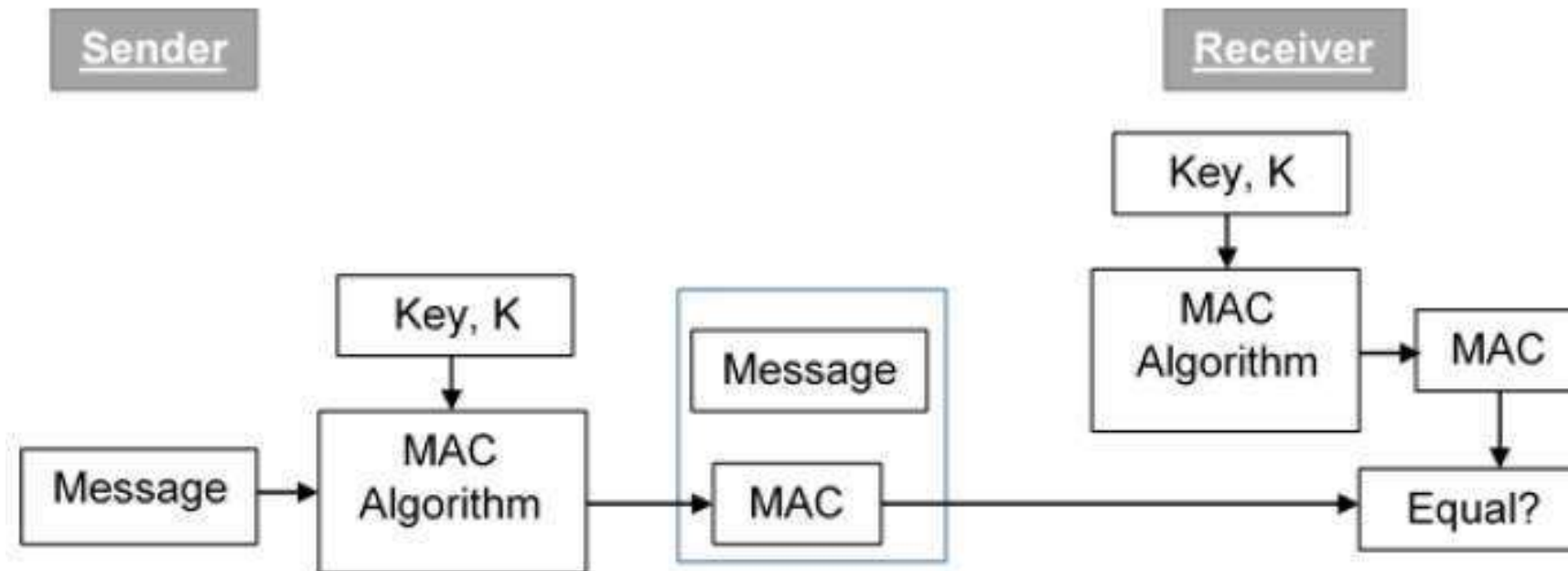
```
public void update(byte[] input); // for  $n-1$  parts
```

```
public byte[] doFinal(byte[] input); // for last part
```



Attach the MAC

- JCA **does not** handle the attachment of the MAC to the message sent on the sender side, nor the verification of the MAC on the recipient side. **It must be handled by the developer.**
- GCM is the only exception (we will see it later).



How to combine Cipher and MAC?

The most common ways to combine Encryption and Mac are:

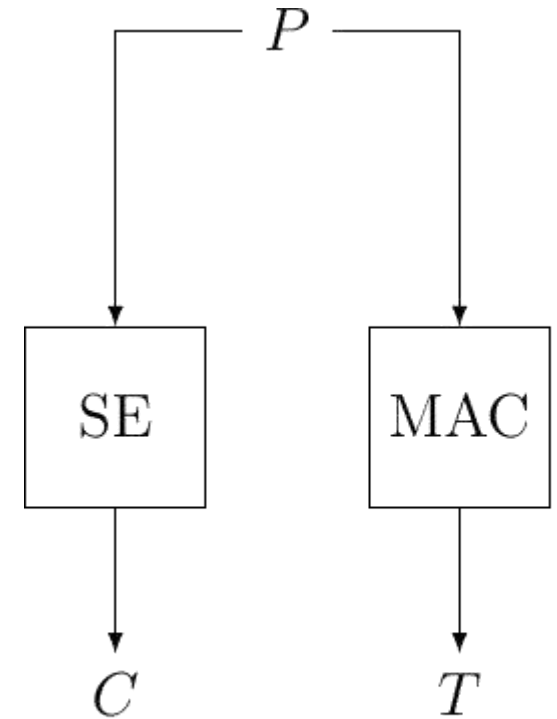
1. Encrypt-and-MAC
2. Mac-then-Encrypt
3. **Encrypt-then-MAC**

Encryption and MAC use two different keys.



Encrypt-and-MAC

- The message is first encrypted, and a MAC is simultaneously calculated on the plaintext. Both are transmitted.
- MAC algorithm is not supposed to maintain confidentiality about the message.
- The same message sent twice will produce the same MAC and this would be visible.
- Encrypt-and-MAC was used in SSH, until version 6.7.
- **DO NOT USE IT!**

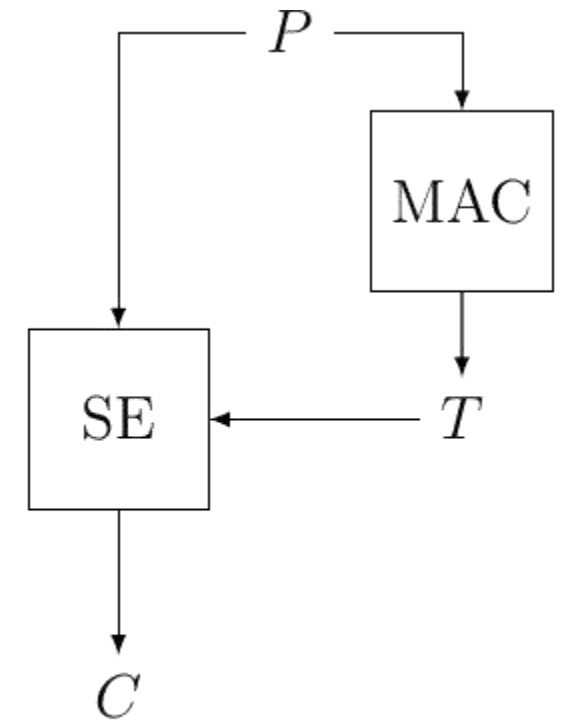


Encrypt-and-MAC



MAC-then-Encrypt

- A MAC is computed on the plaintext, appended, and then the entire package is encrypted.
- *Doom principle*: if you have to perform any cryptographic operation before verifying the MAC on a message you've received, it will somehow inevitably lead to doom.
- The receiver must decrypt the data before verifying the MAC, allowing potential attackers to send arbitrary messages and exploit decryption errors (*padding oracles*).
- Historically used in SSL/TLS before 1.2. **DO NOT USE IT!**



MAC-then-Encrypt



Padding Oracle Attack (1)

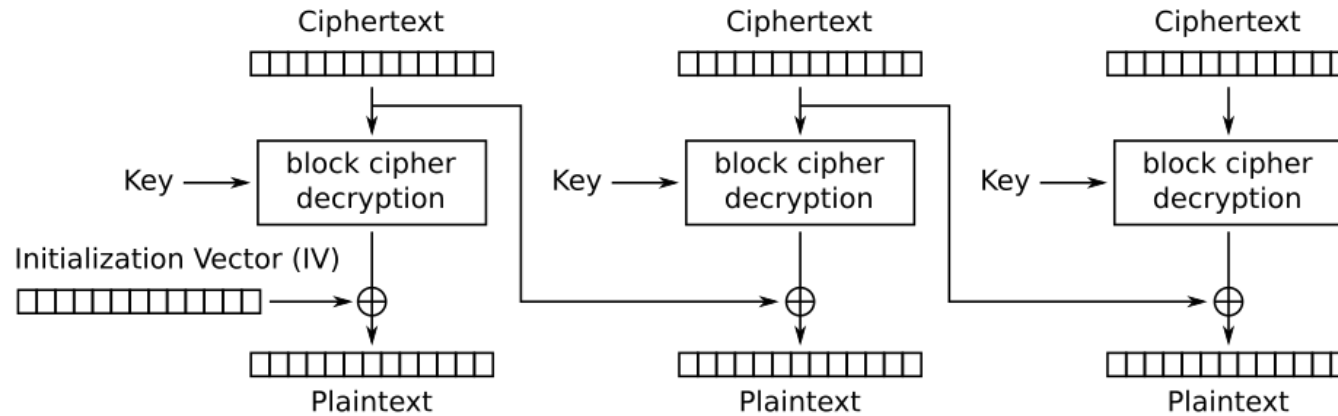
- TLS used MAC-then-encrypt with AES-CBC. *Why is it dangerous?*
- When the block doesn't fit the block size, it is padded.

Byte Nr:	1	2	3	4	5	6	7	8
Text: Exploit	E	x	p	l	o	l	t	0x01
Text: Attack	A	t	t	a	c	k	0x02	0x02
Text: Cyber	C	y	b	e	e	0x03	0x03	0x03
Text: Hack	H	a	c	k	0x04	0x04	0x04	0x04

- What if an attacker can understand if a packet that has been sent is correctly padded or not? The server acts as an **oracle**, and the attacker can exploit this to decrypt the plaintext without knowing the key.



Padding Oracle Attack (2)



Cipher Block Chaining (CBC) mode decryption

$$P_i = D_K(C_i) \oplus C_{i-1}, \text{ if } i \in \{1, N\}$$
$$P_0 = IV \oplus D_K(C_0).$$

and this is true byte by byte.

Suppose the attacker has two ciphertext blocks C_1 , C_2 and wants to decrypt the second block to get plaintext P_2 .

$$P_2 = D_K(C_2) \oplus C_1$$



Padding Oracle Attack (3)

The attacker changes the last byte of C_1 (creating C_1').

The server then returns whether or not the padding of the last decrypted block (P_2') is correct.

The attacker wants to force the last byte to be 0x01.

In fact, since:

$$P_2'[15] = D_K(C_2)[15] \oplus C_1'[15]$$

If the attacker gets a correctly padded block, it can compute $D_K(C_2)[15]$ doing:

$$D_K(C_2)[15] = \underbrace{P_2'[15]}_{0x01} \oplus \underbrace{C_1'[15]}_{\text{Decided by the attacker}}$$

Padding Oracle Attack (4)

- To get $D_K(C_2)[14]$ the attacker should now force 0x02 in the last TWO bytes of P_2' .
- $C_1'[15]$ is set to:

$$C_1'[15] = D_K(C_2)[15] \oplus 0x02$$

- And now the attacker changes $C_1'[14]$ until the padding is valid again.
- Therefore, the attacker can compute $D_K(C_2)[14]$:

$$D_K(C_2)[14] = \underbrace{P_2'[14]}_{0x02} \oplus \underbrace{C_1'[14]}_{\text{Decided by the attacker}}$$

- And so on...



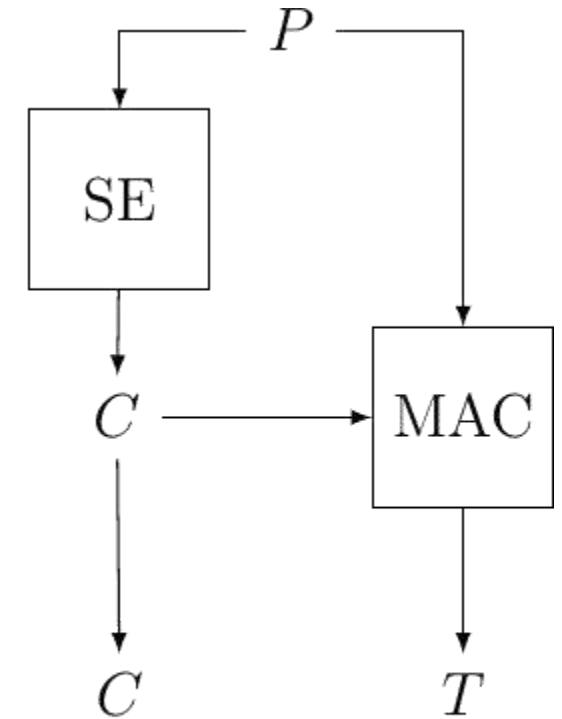
Padding Oracle Attack (5)

- If a block consists of 128 bits (AES, for example), which is 16 bytes, the attacker will obtain plaintext P_2 in no more than $256 \cdot 16 = 4096$ attempts. This is significantly faster than the 2^{128} attempts required to brute-force a 128-bit key.
- What was the problem?
 1. Before checking the MAC, the message is decrypted, then the padding is checked, and, only in the end, the MAC is verified.
 2. The server reveals if the padding is correct (be careful to timing attack also!).



Encrypt then MAC

- Encrypts the plaintext and then calculates a MAC on the ciphertext. **It is considered the most secure approach to authenticated encryption.**
- The receiver verifies the MAC before attempting to decrypt the data. If the tag is invalid, the message is discarded, preventing decryption of manipulated data.
- Common in modern security protocols like IPsec and available as an extension in TLS 1.2 and 1.3.



Encrypt-then-MAC



How to compare MACs

- Before decrypting the message, the receiver **MUST** recalculate the MAC using the same secret key and algorithm as the sender, then perform a constant-time comparison. If the computed MAC matches the received MAC, the message is deemed authentic and untampered.
- **Use Constant-Time Comparison:** When comparing the MACs, use a cryptographic constant-time string comparison function. This prevents *timing attacks*, where an attacker deduces the MAC byte-by-byte based on how long the comparison takes.
- **Avoid Simple Equality Operators:** Do not use standard string comparison methods (like `==` or `Arrays.equals()`) as they often return immediately upon finding the first differing character, creating timing vulnerabilities. **Use `MessageDigest.isEqual()`** .



Xbox360 MAC Timing Attack

- Essentially, an older version of the Xbox kernel had a flaw where `memcmp` was used to check the HMAC value.
- Downgrading to the older kernel version allowed an attacker to provide bogus hashes and measure the difference between a valid byte and invalid one.
- The implementation of MAC verification had a difference of 2.2 milliseconds between rejection times.
- Attackers were able to exploit this and load pirated games onto the hardware.



Example

```
// Encryption
SecureRandom sr = new SecureRandom();

KeyGenerator secretKeyGenerator = KeyGenerator.getInstance("AES");
secretKeyGenerator.init(256, sr);
SecretKey secretKey = secretKeyGenerator.generateKey();

KeyGenerator macKeyGenerator = KeyGenerator.getInstance("HmacSHA256");
macKeyGenerator.init(256, sr);
SecretKey macKey = macKeyGenerator.generateKey();

byte[] iv = new byte[16];
sr.nextBytes(iv);
IvParameterSpec ivSpec = new IvParameterSpec(iv);

Cipher cipher = Cipher.getInstance("AES/CBC/PKCS5Padding");
cipher.init(Cipher.ENCRYPT_MODE, secretKey, ivSpec);

Mac macInstance = Mac.getInstance("HmacSHA256");
macInstance.init(macKey);

byte[] plaintext = "Sicurezza dell'Informazione".getBytes();
byte[] ciphertext = cipher.doFinal(plaintext);
byte[] mac = macInstance.doFinal(ciphertext);
String encodedCiphertext = HexFormat.of().formatHex(ciphertext);
String encodedMac = HexFormat.of().formatHex(mac);
System.out.println("Ciphertext: " + encodedCiphertext);
System.out.println("MAC: " + encodedMac);
```

```
// Decryption
//The receiver already has the keys and the IV
cipher.init(Cipher.DECRYPT_MODE, secretKey, ivSpec);
macInstance.init(macKey);

byte[] computedMac = macInstance.doFinal(ciphertext);
if(!MessageDigest.isEqual(mac, computedMac)) {
    System.out.println(x: "MAC verification failed.");
    return;
}

byte[] decryptedPlaintextBytes = cipher.doFinal(ciphertext);
String decryptedPlaintext = new String(decryptedPlaintextBytes);
System.out.println("Decrypted Plaintext: " + decryptedPlaintext);
```

```
Ciphertext: 96072fc8079abf9c67af40b3a7fd901df7c7791eefcb5cf2ce6f9b1207774a61
MAC: 1917e4f5777535599840fceaafafde5fa1caea1f3ec604d6ef6d6dfdfa52f8ea94f
Decrypted Plaintext: Sicurezza dell'Informazione
```

AEAD (GCM)

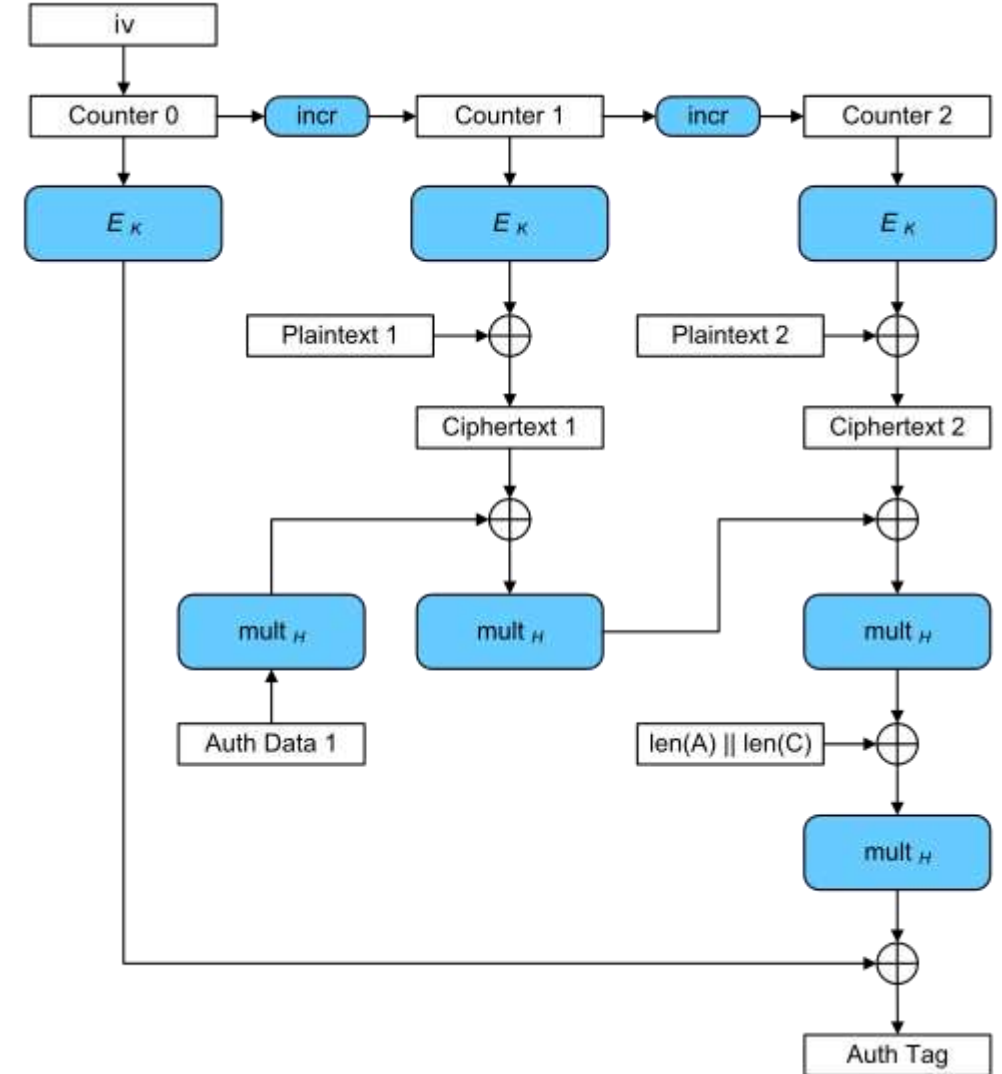
- The **GCM** operating mode belong to the category of Authenticated Encryption with Additional Data ciphers (AEAD) as it leverages a MAC to grant integrity. This cipher not only grant the authenticity of the **ciphertext**, but also of some **additional plaintext data**.
- To authenticate additional (non-encrypted) data, we can use the following function:

```
public final void updateAAD(byte[] src);
```
- Of course, this function must be called both when encrypting, and decrypting, in the same order.
- Widely used in modern standards



GCM

- **GCM** is encrypt-then-mac.
- `Cipher.doFinal()` automatically encrypts data and generates a MAC on the sender side, verifies the MAC and decrypts data on the receiver side.
- Launches `AEADBadTagException` if Mac verification fails.



GCM Example

```
try {  
    // Encryption  
    SecureRandom sr = new SecureRandom();  
    KeyGenerator kg = KeyGenerator.getInstance("AES");  
    kg.init(256, sr);  
    SecretKey key = kg.generateKey();  
  
    byte[] iv = new byte[16];  
    sr.nextBytes(iv);  
    GCMParameterSpec paramSpec = new GCMParameterSpec(128, iv);  
  
    Cipher cipher = Cipher.getInstance("AES/GCM/NoPadding");  
    cipher.init(Cipher.ENCRYPT_MODE, key, paramSpec);  
  
    byte[] plaintext1 = "Sicurezza ".getBytes();  
    byte[] plaintext2 = "dell'".getBytes();  
    byte[] plaintext3 = "Informazione".getBytes();  
    byte[] AAD = "Prof: Rebecca Montanari".getBytes();  
    cipher.updateAAD(AAD);  
    cipher.update(plaintext1);  
    cipher.update(plaintext2);  
    byte[] ciphertext = cipher.doFinal(plaintext3);  
    String encodedCiphertext = HexFormat.of().formatHex(ciphertext);  
    System.out.println("Ciphertext: " + encodedCiphertext);  
  
    // Decryption  
    cipher.init(Cipher.DECRYPT_MODE, key, paramSpec);  
    cipher.updateAAD(AAD);  
    //ciphertext[0] ^= 1;  
    byte[] decryptedPlaintextBytes = cipher.doFinal(ciphertext);  
    String decryptedPlaintext = new String(decryptedPlaintextBytes);  
    System.out.println("Decrypted Plaintext: " + decryptedPlaintext);  
}
```

```
Ciphertext: c78b885d58db64ab36e1785e445cc8aee13ddef466b1e1d479400ab836326d7806b81c79faf705688e855a  
Decrypted Plaintext: Sicurezza dell'Informazione
```

```
Ciphertext: 68c5ef9fa735e8bc43144f4d484a2d90ddd640b39fb0afdef9f1d6fc1d8620819a3c4d0ce125f29df333ca  
Authentication failed!
```

```
} catch (AEADBadTagException e) {  
    System.out.println(x: "Authentication failed!");  
}
```



[https://github.com/alexmark22/EsercitazioniUNIBO-](https://github.com/alexmark22/EsercitazioniUNIBO-SicurezzaDellInformazione)
SicurezzaDellInformazione



Bibliografia

- [1]: JCA Reference Guide - <https://docs.oracle.com/en/java/javase/11/security/java-cryptography-architecture-jca-reference-guide.html>
- [2]: JCA Standard Algorithm Names - <https://docs.oracle.com/javase/8/docs/technotes/guides/security/StandardNames.html>
- [3]: IV mishandling – <https://stackerred.com/blog/iv-mishandling/>
- [4]: Some cryptographic attacks – <https://research.checkpoint.com/2024/modern-cryptographic-attacks-a-guide-for-the-perplexed/>
- [5]: BEAST Attack – <https://blog.cryptographyengineering.com/2011/09/21/brief-diversion-beast-attack-on-tlsssl/>
- [6]: Crib dragging - <https://samwho.dev/blog/toying-with-cryptography-crib-dragging/>
- [7]: Padding Oracle Attack - <https://blog.cloudflare.com/padding-oracles-and-the-decline-of-cbc-mode-ciphersuites/>



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

Alessandro Marchesano

alessandr.marchesano@unibo.it

www.unibo.it